ЛАБОРАТОРНА РОБОТА №4

Вдосконалення структури коду графічного редактора об’єктів на C++

Дисципліна	Об’єктно-орієнтоване програмування
Студент	Чакун Олександр Сергійович
Група	ІМ-о51
Номер у списку / варіант	16
Викладач	Рекечинський Дмитро Олександрович

2026

1. Мета роботи

Отримати практичні навички проєктування класів та рефакторингу об’єктно-орієнтованої програми, зменшити залежність редактора від конкретних типів фігур, реалізувати динамічне керування об’єктами та множинне успадкування.

2. Варіант та вимоги

Номер варіанта дорівнює номеру студента у списку: 16. Оскільки номер парний, у програмі реалізовано динамічний об’єкт MyEditor.

Кольори та стилі чотирьох базових фігур збережено за попередньою Lab3. Гумовий слід під час введення всіх фігур малюється пунктирною лінією.

Крім PointShape, LineShape, RectShape та EllipseShape додано два нові типи: LineOOShape (лінія з кружечками на кінцях) та CubeShape (каркас куба). Обидва нові типи реалізовано через множинне успадкування.

Toolbar містить шість кнопок з підказками (tooltips) для всіх типів фігур.

3. Структура програми та рефакторинг

У попередніх роботах графічний редактор мав окремі методи запуску для кожного типу фігури. У Lab4 цю залежність замінено одним універсальним методом MyEditor::Start(Shape*). Тому клас MyEditor працює через абстракцію Shape і не потребує окремого Start-методу для кожного нового типу.

```
class MyEditor
{
private:
    Shape** shapes_{ nullptr };
    Shape* prototype_{ nullptr };
    Shape* working_{ nullptr };
    ...
public:
    MyEditor() = default;
    ~MyEditor();
    void Start(Shape* shape);
    void OnLBdown(HWND hWnd);
    void OnLBup(HWND hWnd);
    void OnMouseMove(HWND hWnd);
    void OnPaint(HWND hWnd, HDC hdc);
};
```

3.1. Динамічний MyEditor

Для парного варіанта №16 редактор створюється динамічно при WM_CREATE і знищується при WM_DESTROY.

```
static MyEditor* g_editor = nullptr;

case WM_CREATE:
    g_editor = new MyEditor();
    g_editor->OnCreate(hWnd);
    g_editor->Start(new PointShape());
    return 0;

case WM_DESTROY:
    delete g_editor;
    g_editor = nullptr;
    PostQuitMessage(0);
    return 0;
```

3.2. Динамічний масив фігур

MyEditor зберігає об’єкти у динамічному масиві Shape**. Масив створюється при першій необхідності і збільшується вдвічі після заповнення поточної місткості.

```
void MyEditor::EnsureCapacity()
{
    if (count_ < capacity_) return;

    const int newCapacity = (capacity_ == 0) ? 16 : capacity_ * 2;
    Shape** newArray = new Shape*[newCapacity]{};

    for (int i = 0; i < count_; ++i)
        newArray[i] = shapes_[i];

    delete[] shapes_;
    shapes_ = newArray;
    capacity_ = newCapacity;
}
```

4. Ієрархія Shape та поліморфізм

Shape є абстрактним базовим класом. Метод Show є чисто віртуальним, тому кожна конкретна фігура реалізує власне відображення. Clone дає змогу MyEditor створити копію активного типу без перевірок конкретного класу.

```
class Shape
{
protected:
    long xs1_, ys1_, xs2_, ys2_;
public:
    virtual ~Shape() = default;
    void Set(long x1, long y1, long x2, long y2);
    virtual void Show(HDC hdc) = 0;
    virtual Shape* Clone() const = 0;
    virtual const wchar_t* Name() const = 0;
};
```

5. Множинне успадкування

Нові фігури побудовано на функціональності вже існуючих класів. Для уникнення дублювання базового Shape у ромбічній ієрархії LineShape, RectShape та EllipseShape успадковують Shape віртуально.

```
class LineShape    : virtual public Shape { ... };
class RectShape    : virtual public Shape { ... };
class EllipseShape : virtual public Shape { ... };

class LineOOShape : public LineShape, public EllipseShape { ... };
class CubeShape   : public LineShape, public RectShape    { ... };
```

5.1. LineOOShape

LineOOShape викликає LineShape::Show для основної лінії і EllipseShape::Show для двох кружечків на її кінцях.

```
void LineOOShape::Show(HDC hdc)
{
    ...
    LineShape::Show(hdc);

    Set(x1 - r, y1 - r, x1 + r, y1 + r);
    EllipseShape::Show(hdc);

    Set(x2 - r, y2 - r, x2 + r, y2 + r);
    EllipseShape::Show(hdc);
    ...
}
```

5.2. CubeShape

CubeShape формує дві прямокутні грані через RectShape::Show і з’єднує відповідні вершини через LineShape::Show.

```
Set(x1, y1, x2, y2);
RectShape::Show(hdc);

Set(x1 + dx, y1 + dy, x2 + dx, y2 + dy);
RectShape::Show(hdc);

Set(x1, y1, x1 + dx, y1 + dy);
LineShape::Show(hdc);
...
```

6. Пунктирний гумовий слід

Під час введення тимчасова фігура малюється поліморфно через Show. Для варіанта Lab4 використано пунктирне перо PS_DOT.

```
HPEN pen = CreatePen(PS_DOT, 1, RGB(0, 0, 0));
...
working_->SetPreview(true);
working_->Show(hdc);
working_->SetPreview(false);
```

7. Toolbar та tooltips

Toolbar містить шість команд: крапка, лінія, прямокутник, еліпс, лінія з кружечками та каркас куба. Підказки для кнопок формуються у MyEditor::OnNotify при повідомленні TTN_GETDISPINFOW.

8. Діаграма класів

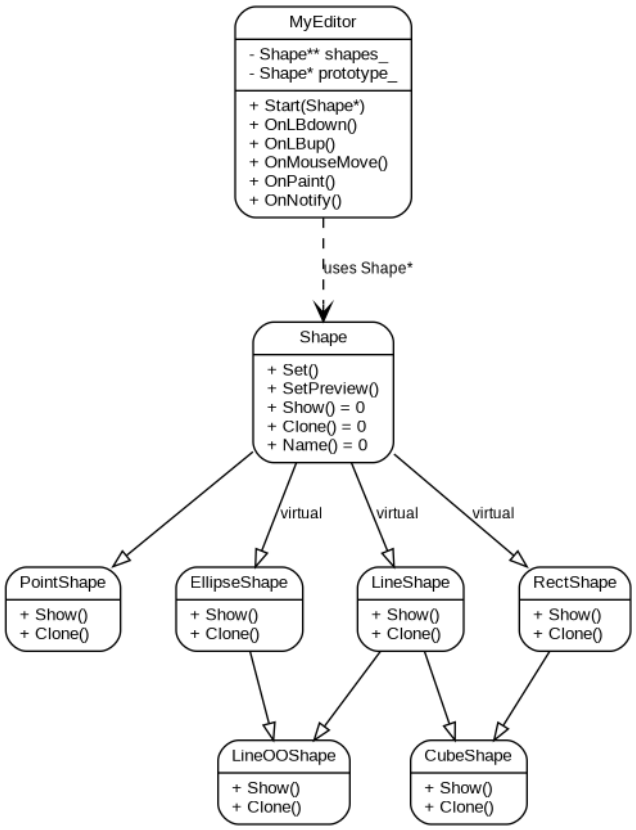


Рис. 1. Діаграма класів Lab4

9. Результати роботи програми

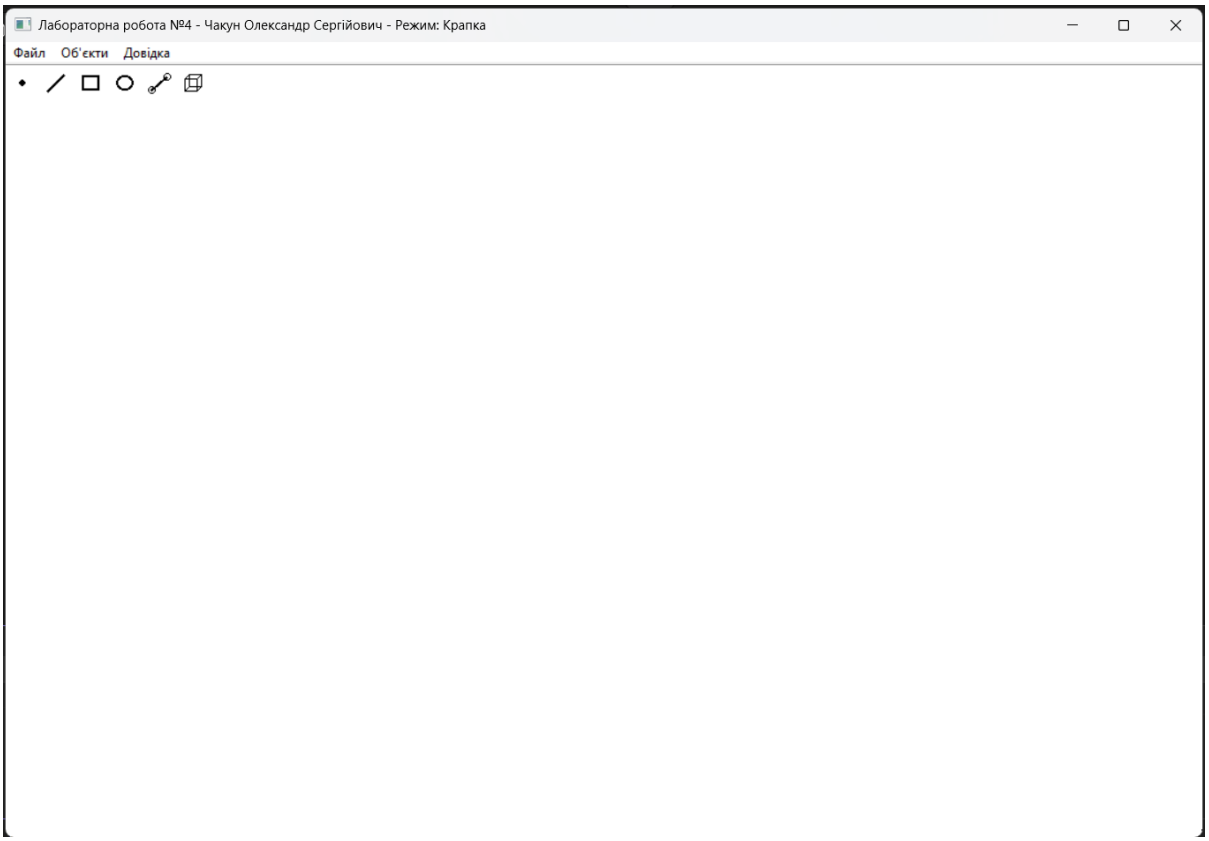


Рис. 2. Головне вікно програми після запуску

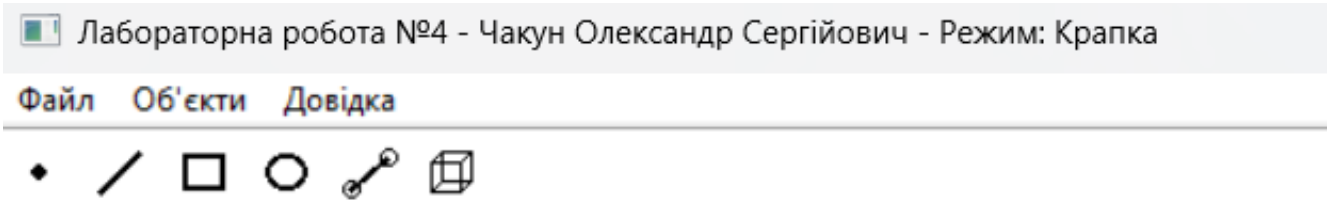


Рис. 3. Toolbar з шістьма кнопками

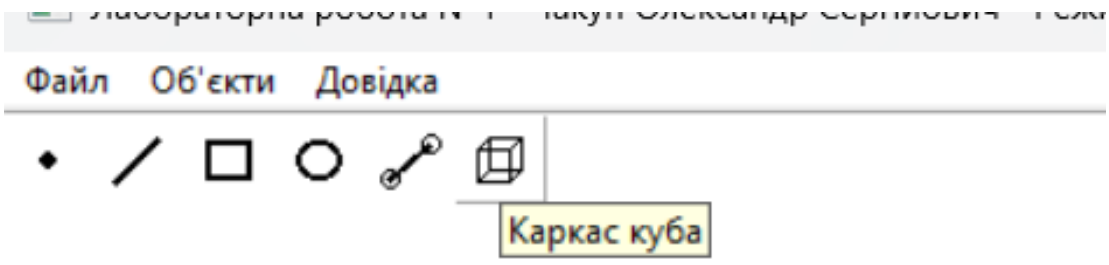


Рис. 4. Підказка для кнопки «Каркас куба»

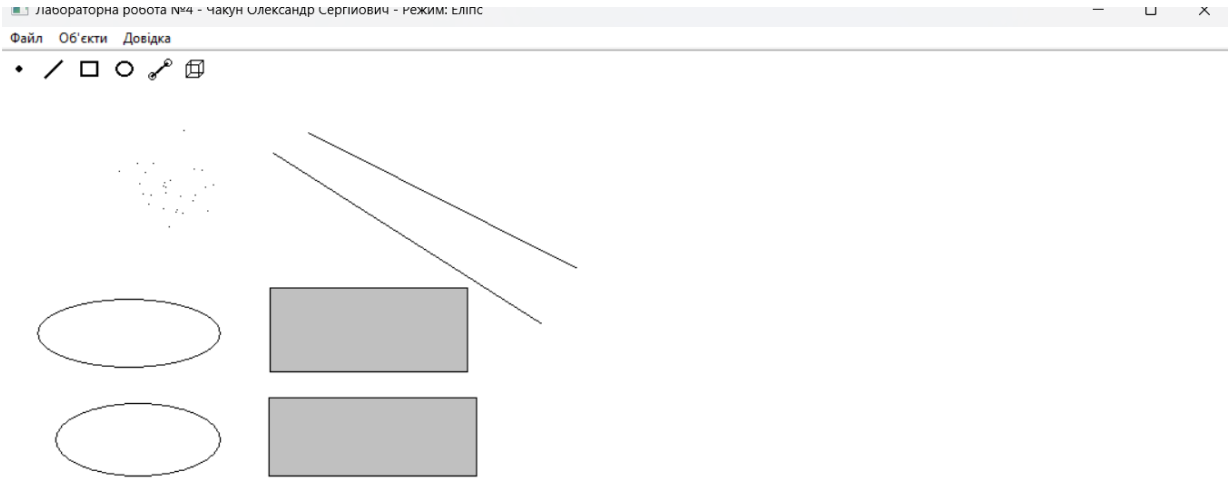


Рис. 5. Базові фігури: лінії, прямокутники та еліпси

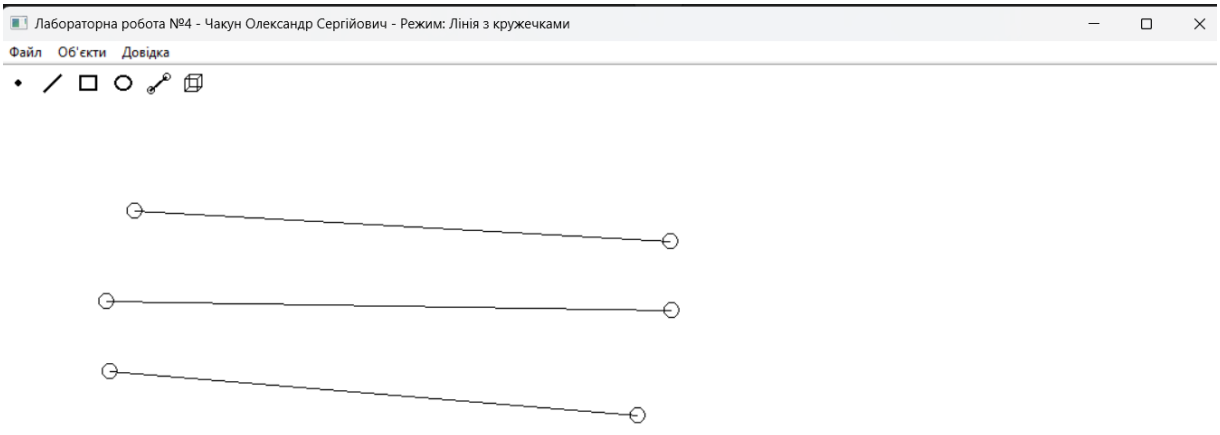


Рис. 6. Об’єкти LineOOShape - лінії з кружечками на кінцях

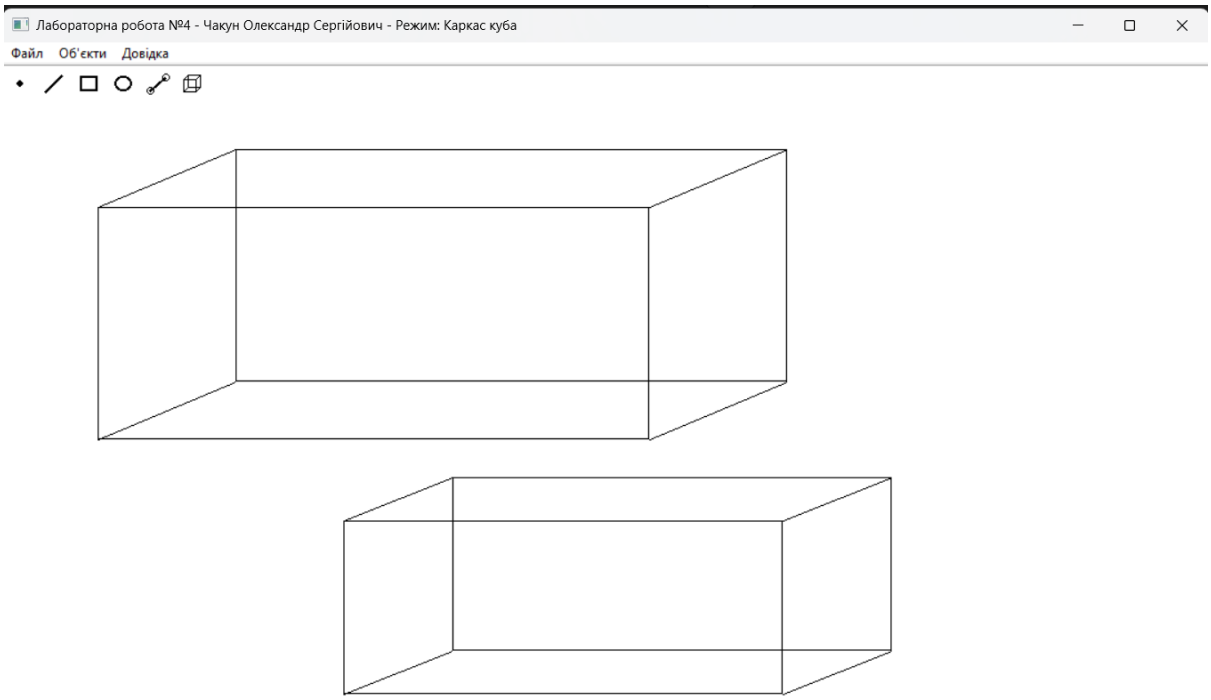


Рис. 7. Об’єкти CubeShape - каркас куба

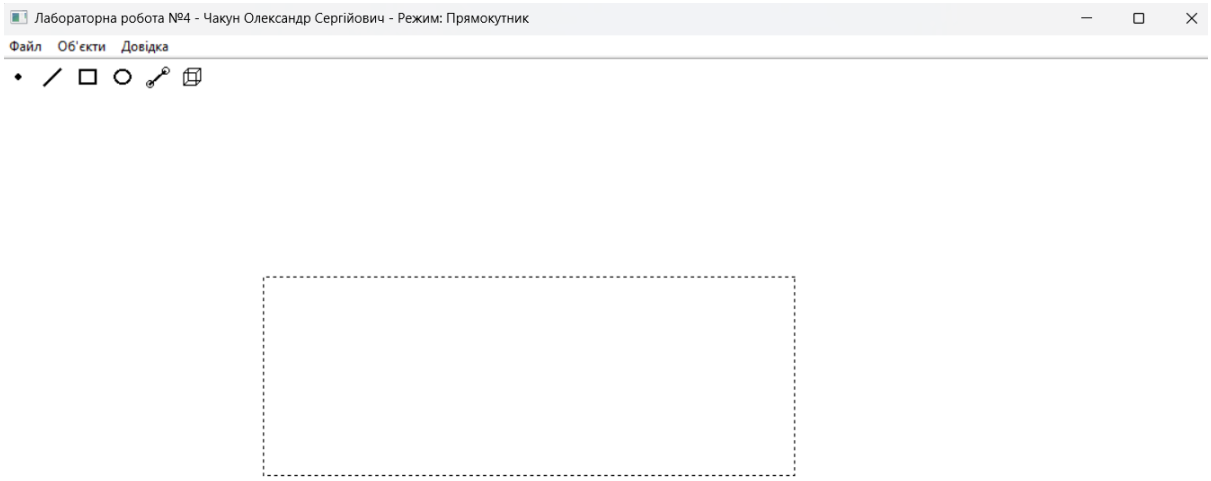


Рис. 8. Пунктирний «гумовий» слід під час введення прямокутника

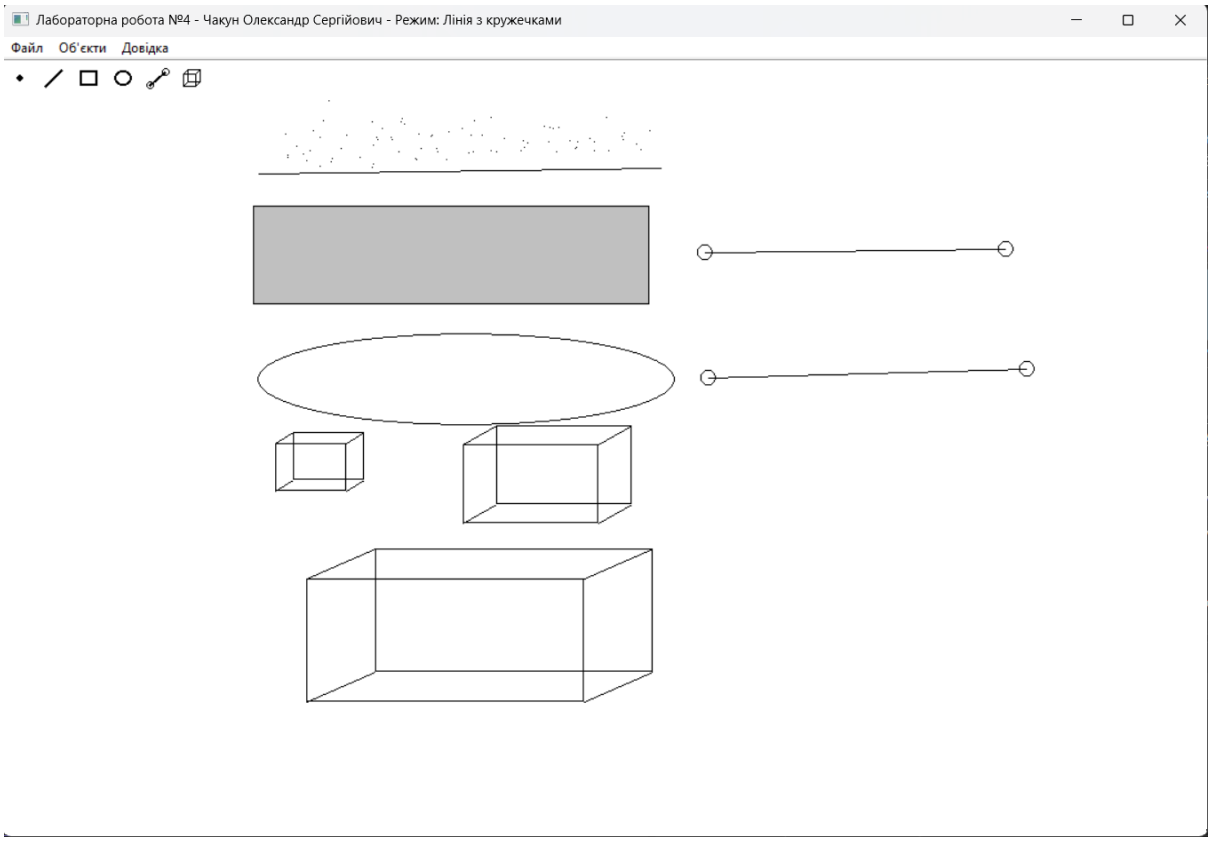


Рис. 9. Результат роботи редактора з різними типами фігур

10. Висновки

У лабораторній роботі виконано рефакторинг графічного редактора. Клас MyEditor тепер залежить від абстракції Shape та використовує один універсальний метод Start(Shape*), що спрощує додавання нових типів фігур. Для варіанта №16 реалізовано динамічне створення і знищення MyEditor та динамічний масив Shape**. Нові фігури LineOOShape і CubeShape реалізовано через множинне успадкування, а віртуальне успадкування Shape усуває дублювання базової частини у ромбічній ієрархії. Toolbar містить шість кнопок з підказками, а гумовий слід усіх фігур відображається пунктирною лінією. Програма успішно запускається та демонструє роботу всіх реалізованих типів об’єктів.